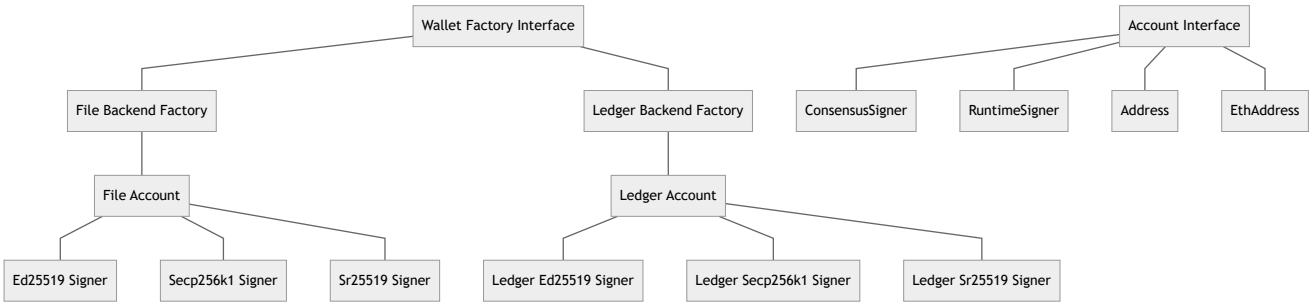
지갑 시스템 상세 분석 문서

개요

Oasis CLI의 지갑 시스템은 다양한 암호화 알고리즘과 백엔드를 지원하는 유연하고 확장 가능한 아키텍처를 제공합니다. 이 시스템은 팩토리 패턴을 사용하여 파일 기반 지갑과 Ledger 하드웨어 지갑을 통합적으로 관리하며, 각각 고유한 보안 특성과 사용 사례를 가지고 있습니다.

지갑 시스템 아키텍처

핵심 구조



팩토리 패턴 구현

지갑 시스템의 핵심은 `Factory` 인터페이스로, 각 백엔드 타입(파일, Ledger)에 대해 구체적인 구현을 제공합니다:

```
type Factory interface {
Kind() string  // 백엔드 종류 반환
PrettyKind() string  // 사용자 친화적 이름
Create() (Account, error) // 새 계정 생성
Load() (Account, error) // 기존 계정 로드
Import() (Account, error) // 계정 가져오기
Remove() error  // 계정 삭제
Rename() error  // 계정 이름 변경
// ... 기타 메서드
}
```

이 패턴의 장점:

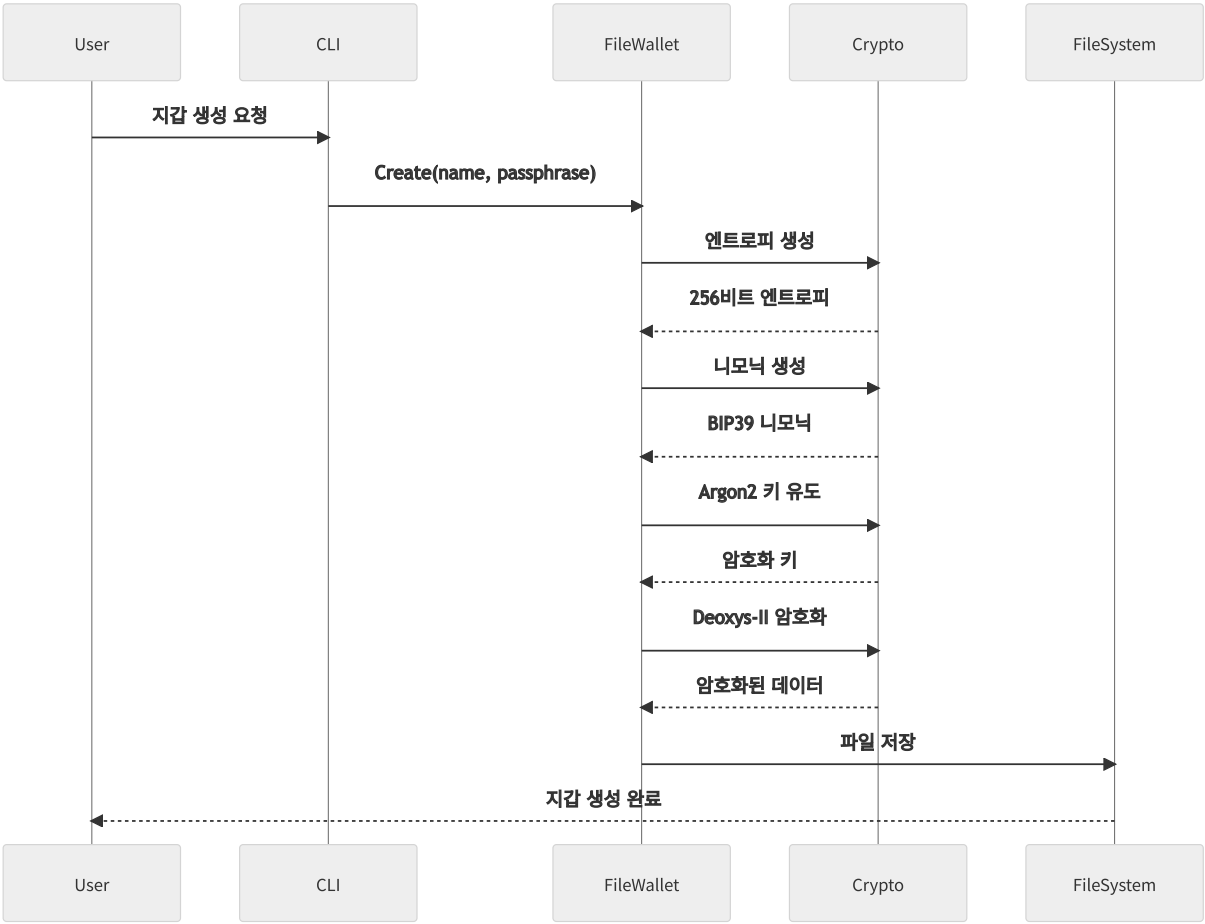
- **확장성**: 새로운 백엔드 타입을 쉽게 추가 가능
- **일관성**: 모든 백엔드가 동일한 인터페이스 제공
- **유연성**: 런타임에 백엔드 선택 가능

파일 기반 지갑 시스템

구조 및 특징

파일 기반 지갑은 암호화된 키 파일을 로컬 파일 시스템에 저장하는 방식입니다.

보안 구조



암호화 메커니즘

키 유도 함수 (KDF)

- **알고리즘**: Argon2id
- **매개변수**:
- Time: 1 iteration

- Memory: 64KB
- Threads: 4
- Salt: 32바이트 랜덤
- Output: 32바이트 키

대칭 암호화

- **알고리즘**: Deoxys-II (AEAD)
- **키 크기**: 256비트
- **논스**: 32바이트 랜덤

파일 구조

```
{
"kdf": {
"argon2": {
"salt": "base64-encoded-salt",
"time": 1,
"memory": 65536,
"threads": 4
}
},
"nonce": "base64-encoded-nonce",
"data": "encrypted-secret-data"
}
```

지원되는 암호화 알고리즘

1. Ed25519 계열

Ed25519-ADR8

- **용도**: Oasis 합의 계층 및 노드 운영자
- **키 유도**: ADR-8 표준 (BIP-32 기반)
- **경로**: `m/44'/474'/{account_number}'`
- **특징**:
- 합의 계층 서명자 지원
- 결정론적 키 생성
- 높은 보안성

Ed25519-Raw

- **용도**: 직접 개인키 가져오기
- **키 유도**: 없음 (원시 키 사용)
- **인코딩**: Base64
- **특징**:
- 기존 키 직접 사용
- 빠른 로딩
- 백업 복원 용이

Ed25519-Legacy

- **용도**: 기존 ROSE 코인 타입 호환성
- **키 유도**: 레거시 경로
- **경로**: `m/44'/474'/0'/0'/{key_number}'`

2. Secp256k1 계열

Secp256k1-BIP44

- **용도**: Ethereum 호환 및 Sapphire ParaTime
- **키 유도**: BIP-44 표준
- **경로**: `m/44'/60'/0'/0/{account_number}`
- **특징**:
- Ethereum 주소 생성
- EIP-55 체크섬 지원
- 메타마스크 호환성

Secp256k1-Raw

- **용도**: 직접 개인키 가져오기
- **키 유도**: 없음
- **인코딩**: 16진수 (0x 접두사 선택적)
- **크기**: 32바이트

3. Sr25519 계열

Sr25519-ADR8

- **용도**: Schnorrkel/Ristretto x25519 서명
- **키 유도**: ADR-8 호환 경로
- **특징**:
- 배치 검증 지원
- 높은 성능
- Substrate 호환성

Sr25519-Raw

- **용도**: 직접 개인키 가져오기
- **인코딩**: Base64
- **크기**: 32바이트

키 유도 구현

ADR-8 기반 유도 (Ed25519)

```
func Ed25519FromMnemonic(mnemonic string, number uint32) (Signer, []byte, err
// SAKG (Staking Account Key Generation) 사용
signer, _, err := sakg.GetAccountSigner(mnemonic, "", number)
if err != nil {
return nil, nil, err
}
return ed25519.WrapSigner(signer), signer.UnsafeBytes(), nil
}
```

BIP-44 기반 유도 (Secp256k1)

```
func Secp256k1FromMnemonic(mnemonic string, number uint32) (Signer, []byte, e
wallet, err := hdwallet.NewFromMnemonic(mnemonic)
if err != nil {
return nil, nil, err
}
```

```
path := fmt.Sprintf("m/44'/60'/0'/0/%d", number)

derivationPath := hdwallet.MustParseDerivationPath(path)

account, err := wallet.Derive(derivationPath, false)

if err != nil {

return nil, nil, err

}

privateKey, err := wallet.PrivateKeyBytes(account)

if err != nil {

return nil, nil, err

}

return secp256k1.NewSigner(privateKey), privateKey, nil

}
```

계정 가져오기 지원

니모닉 가져오기

- **지원 알고리즘**: ADR-8, BIP-44 기반 알고리즘
- **검증**: BIP-39 표준 준수 확인
- **복구**: 키 번호 지정으로 다중 계정 복구

개인키 가져오기

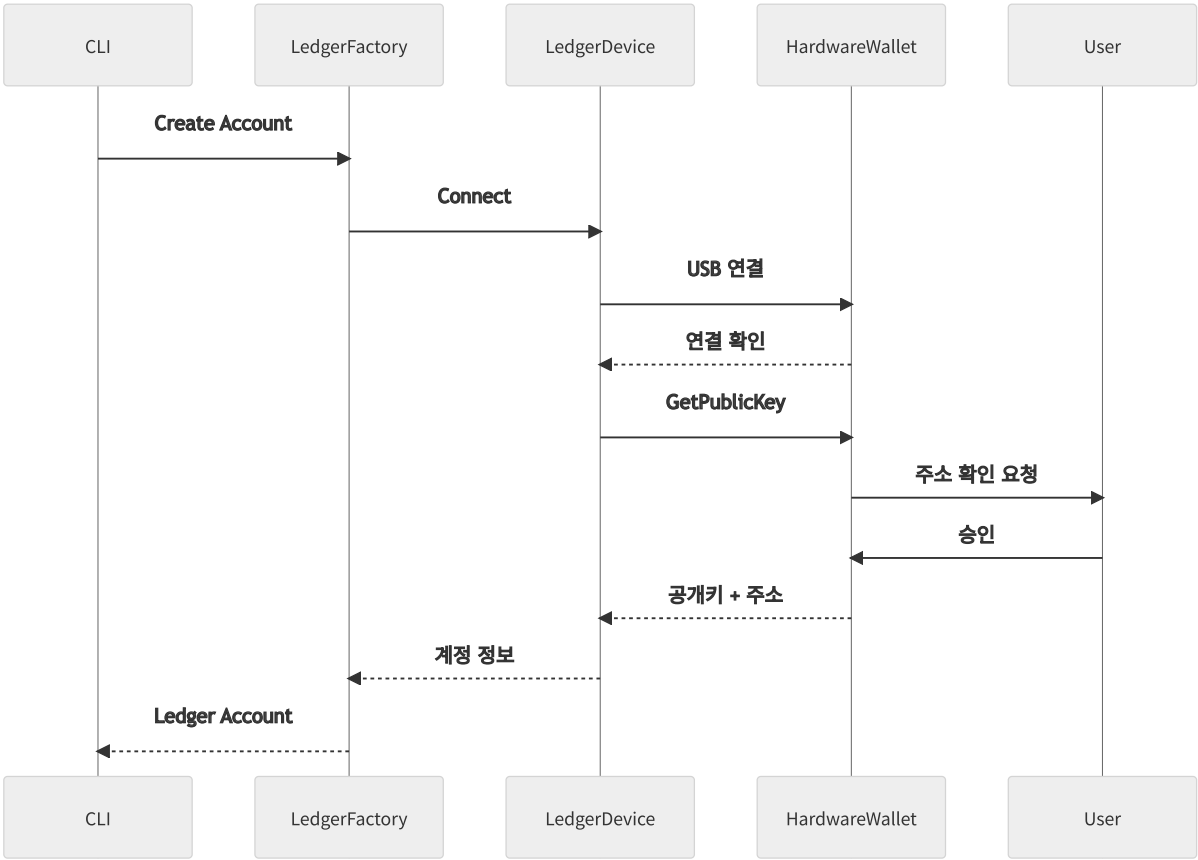
- **지원 알고리즘**: Raw 알고리즘
- **형식 검증**: 알고리즘별 인코딩 확인
- **보안**: 메모리에서 즉시 암호화

Ledger 하드웨어 지갑 시스템

구조 및 특징

Ledger 지갑은 하드웨어 보안 모듈을 사용하여 개인키를 안전하게 보관하고 서명을 수행합니다.

연결 아키텍처



지원되는 암호화 알고리즘

Ed25519 (ADR-8 및 Legacy)

- **경로**: ADR-8 `m/44'/474'/{number}'` 또는 Legacy `m/44'/474'/0'/0'/{number}'`
- **용도**: Oasis 합의 계층
- **특징**: 하드웨어에서 직접 서명

Secp256k1 (BIP-44)

- **경로**: `m/44'/60'/0'/0/{number}`
- **용도**: Ethereum 호환 트랜잭션
- **특징**: EIP-155 서명 지원

Sr25519 (ADR-8)

- **경로**: `m/44'/474'/{number}'`
- **용도**: Schnorrkel 서명
- **특징**: 배치 검증 최적화

디바이스 통신 프로토콜

APDU 명령어 구조

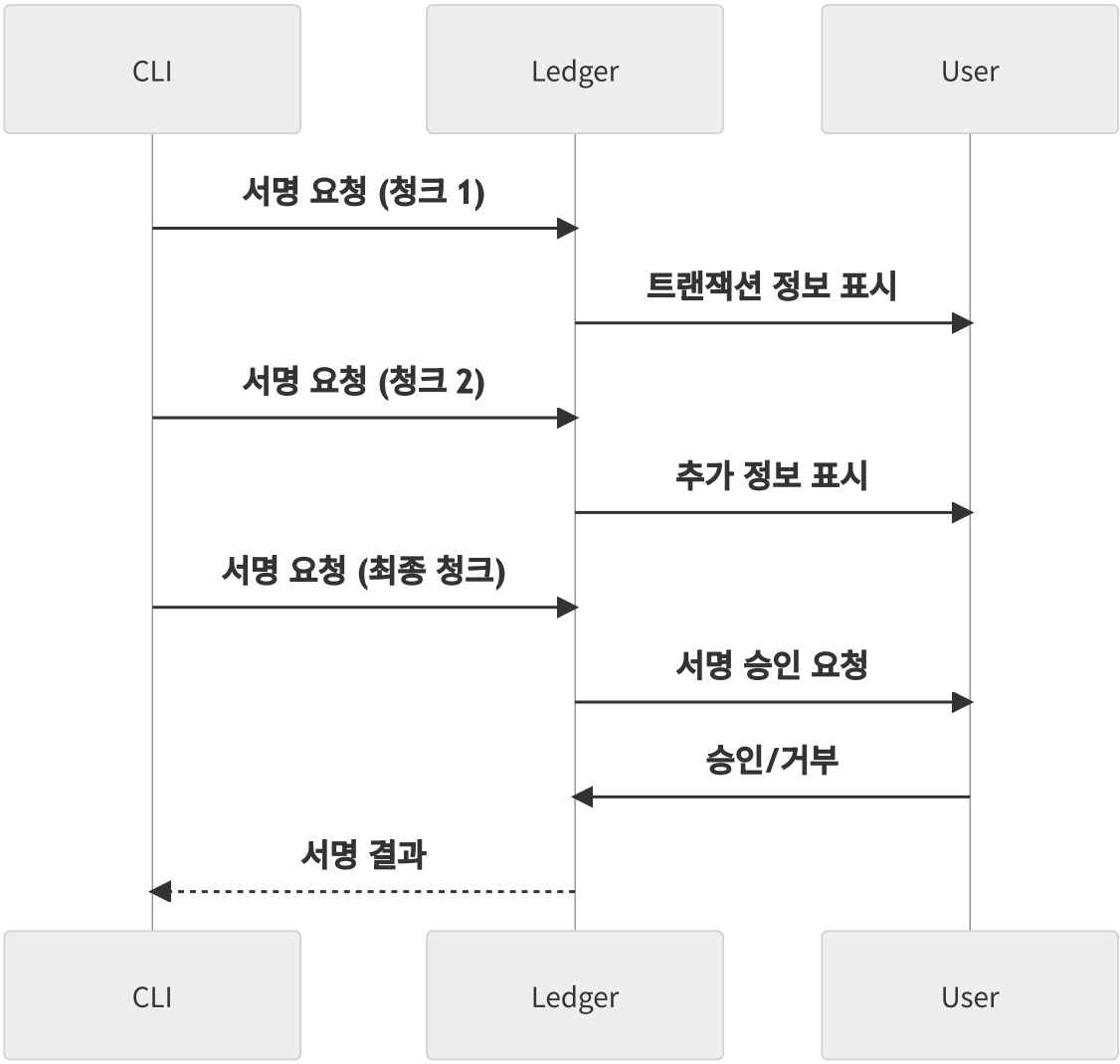
```
CLA | INS | P1 | P2 | LC | DATA | LE
```

주요 명령어:

- `INS_GET_VERSION (0x00)` : 앱 버전 조회
- `INS_GET_ADDR_ED25519 (0x01)` : Ed25519 주소 조회
- `INS_SIGN_ED25519 (0x02)` : Ed25519 서명
- `INS_GET_ADDR_SECP256K1 (0x04)` : Secp256k1 주소 조회

- `INS_SIGN_RT_* (0x05-0x07)` : 런타임 서명

서명 프로세스



보안 특징

하드웨어 보안

- **개인키 격리**: 키가 하드웨어를 벗어나지 않음
- **물리적 확인**: 사용자가 디바이스에서 직접 승인
- **변조 방지**: 하드웨어 레벨 보안

사용자 확인

- **주소 검증**: 생성된 주소를 디바이스에서 확인

- **트랜잭션 검토**: 서명 전 트랜잭션 내용 표시
- **명시적 승인**: 각 작업에 대한 사용자 승인 필요

계정 인터페이스 및 주소 체계

Account 인터페이스

모든 계정 타입은 공통 인터페이스를 구현합니다:

```
type Account interface {
ConsensusSigner() coreSignature.Signer // 합의 계층 서명자
Signer() signature.Signer // 런타임 서명자
Address() types.Address // Oasis 주소
EthAddress() *ethCommon.Address // Ethereum 주소 (해당시)
SignatureAddressSpec() types.SignatureAddressSpec // 서명 주소 스펙
UnsafeExport() (string, string) // 안전하지 않은 내보내기
}
```

주소 생성 방식

Oasis 주소

- **Ed25519**: `oasis1` + Bech32 인코딩
- **Secp256k1**: `oasis1` + Bech32 인코딩 (압축된 공개키 기반)
- **Sr25519**: `oasis1` + Bech32 인코딩

Ethereum 주소 (Secp256k1만 해당)

- **생성**: Keccak256(압축되지 않은 공개키)[12:]
- **형식**: 0x + 40자리 16진수
- **체크섬**: EIP-55 표준

설정 관리 및 마이그레이션

설정 구조

```
[wallet]
default = "my-account"

[wallet.all.my-account]
kind = "file"
address = "oasis1..."

[wallet.all.my-account.config]
algorithm = "ed25519-adr8"
number = 0
```

자동 마이그레이션

Ledger 설정 마이그레이션

1. **1단계**: `derivation` → `algorithm` 변환
2. **2단계**: 기본 알고리즘 설정 (ADR-8)

```go
func (af *ledgerAccountFactory) Migrate(rawCfg map[string]interface{}) bool {
var changed bool
// 기본 알고리즘 설정
if rawCfg["algorithm"] == nil {
rawCfg["algorithm"] = wallet.AlgorithmEd25519Adr8
changed = true
}
// derivation → algorithm 변환
if val, ok := rawCfg["derivation"]; ok {
rawCfg["algorithm"] = fmt.Sprintf("ed25519-%s", val)
delete(rawCfg, "derivation")
changed = true
}
return changed
}
```

에러 처리 및 복구

일반적인 에러 상황

파일 지갑 에러

- **잘못된 비밀번호**: “failed to open account state (maybe incorrect passphrase?)”
- **손상된 파일**: JSON 파싱 에러 또는 복호화 실패

- **권한 문제**: 파일 읽기/쓰기 권한 부족

Ledger 지갑 에러

- **디바이스 미연결**: “no devices connected”
- **사용자 거부**: “signing request rejected by user”
- **앱 미설치**: APDU 통신 실패
- **다중 디바이스**: “multiple devices not supported”

복구 전략

자동 복구

- **연결 재시도**: 네트워크 오류 시 자동 재시도
- **설정 마이그레이션**: 구버전 설정 자동 업데이트
- **메모리 정리**: 민감한 데이터 자동 삭제

수동 복구

- **니모닉 복구**: BIP-39 니모닉으로 계정 복구
- **개인키 가져오기**: 백업된 개인키로 복구
- **설정 재생성**: 손상된 설정 파일 재생성

성능 최적화

메모리 관리

- **즉시 삭제**: 사용 후 민감한 데이터 즉시 삭제
- **제로화**: 메모리 영역을 0으로 덮어쓰기
- **가비지 컬렉션**: Go GC와 협력하여 메모리 정리

연결 최적화

- **연결 재사용**: Ledger 디바이스 연결 재사용
- **배치 처리**: 여러 작업을 배치로 처리
- **타임아웃 관리**: 적절한 타임아웃 설정

확장성 및 유지보수성

새로운 백엔드 추가

1. **Factory 구현**: `wallet.Factory` 인터페이스 구현
2. **Account 구현**: `wallet.Account` 인터페이스 구현
3. **등록**: `wallet.Register()` 함수로 등록

```
func init() {
wallet.Register(&newBackendFactory{})
}
```

새로운 알고리즘 지원

1. **서명자 구현**: 해당 알고리즘의 서명자 구현
2. **키 유도**: 니모닉/개인키에서 키 유도 로직
3. **주소 생성**: 알고리즘별 주소 생성 로직
4. **테스트**: 포괄적인 단위 테스트 작성

보안 고려사항

키 관리

- **메모리 보안**: 민감한 데이터의 메모리 노출 최소화
- **파일 권한**: 지갑 파일의 적절한 권한 설정 (0600)
- **암호화 강도**: 업계 표준 암호화 알고리즘 사용

사용자 인터페이스

- **비밀번호 정책**: 강력한 비밀번호 권장
- **확인 절차**: 중요한 작업에 대한 이중 확인
- **에러 메시지**: 보안에 민감한 정보 노출 방지

이 지갑 시스템은 보안성, 유연성, 확장성을 모두 고려하여 설계되었으며, 다양한 사용 사례와 보안 요구사항을 충족할 수 있는 강력한 기반을 제공합니다.